

OBLIG 2 IN2010

Av: Frida Berge

- Deloppgave 1**
- Deloppgave 2**
- Deloppgave 3**
 - Figurer**

Deloppgave 1:

Implementerte sorteringsalgoritmer:

- Insertion sort
- Quick sort
- Selection sort
- Merge sort

Testet alle fire sorteringsalgoritmene med `nearly_sorted_10` og `random_10`. Alle algoritmene ga rimelige svar (sortert output i stigende rekkefølge). Valgte de to korteste input-ene som tester fordi det går raskt å se med et overblikk at sorteringen er riktig.

Deloppgave 2:

Brukte skallet vi fikk utdelt med lenke til, i GitHub. Gjorde kun endringer ved å legge til de to valgfrie metodene, og deres resultater. Byttene og sammenlikningene er målt ved at jeg brukte metodene i klassen 'Sorter.java'. For hver if-test eller while-løkke hvor tall ble sammenliknet brukte jeg metoden $lt(a < b)$, $leq(a \leq b)$, $gt(a > b)$, $geq(a \geq b)$ eller $eq(a == b)$. og alle steder i koden der to verdier i lista skulle bytte plass brukte jeg metoden $swap(i, j)$. Disse metodene gjorde at verdiene 'swaps' og 'comparisons' økte med én (som en teller). Disse verdiene ble videre brukt til å lage en csv-fil med resultater.

I sorteringsalgoritmen merge foregår det ingen interne bytter i den originale lista fordi sorteringen skjer eksternt. Jeg valgte likevel å øke swap-telleren for hver gang det blir lagt til et tall i den originale lista A for å vise at det blir gjort endringer i lista. Metoden merge kunne også vært optimalisert med tanke på kjøretid ved at jeg hadde brukt den innebygde metoden `Arrays.copyOfRange(A, 0, midten)` to ganger når lista skulle splittes i stedet for en for-løkke slik jeg gjorde.

I tillegg kan det være uklart hva som regnes som en sammenlikning. Jeg valgte å telle absolutt alle sammenlikninger (if og while). Hvis jeg hadde valgt å kun regne med de sammenlikningene som sjekker tall i selve lista A, hadde antallet drastisk minnet. For eksempel i sorteringsalgoritmen insertion valgte jeg å bruke sammenliknings-metodene fra `Sorter.java` i while-løkken. Her kunne jeg for eksempel halvert `insertion.cmp`, ved å droppe den ene sammenlikningen i løkken.

Deloppgave 3:

- Beskrivelse av eksperiment:

- Kompilerte og kjørte java-filene med ulike input
- Brukte output-filene i CSV-format til å legge inn data i Excel og lagde passende diagrammer
- Studerte de ulike diagrammene for å trekke konklusjoner

- I hvilken grad stemmer kjøretiden overens med kjøretidsanalysene (store O) for de ulike algoritmene?

- Kjøretidsanalyse:
 - Insertion – $O(n^2)$
 - Ja, den følger kvadratisk tid (se Figur 1)
 - Kan konkludere med dette fordi grafen er formet som en karakteristisk kvadratisk funksjon. $O(n^2)$ er beregnet ved verste tilfelle, derfor kan man ikke se på grafen at en gitt verdi n på x-aksen tilsvarer en gitt kjøretid lik n^2 på y-aksen
 - Selection – $O(n^2)$
 - Ja, den følger kvadratisk tid (se Figur 1)
 - Kan konkludere med dette fordi grafen er formet som en karakteristisk kvadratisk funksjon. $O(n^2)$ er beregnet ved verste tilfelle, derfor kan man ikke se på grafen at en gitt verdi n på x-aksen tilsvarer en gitt kjøretid lik n^2 på y-aksen
 - Merge – $O(n \cdot \log(n))$
 - Ja, den følger logaritmisk tid (se Figur 2)
 - Kan antyde ved et raskt overblikk at: ved $n = 1000$ er kjøretiden $O = 1000 \cdot \log(1000) = 1000 \cdot 3 = 3000$
 - Quick – $O(n \cdot \log(n))$
 - Ja, den følger logaritmisk tid (se Figur 2)
 - Kan antyde ved et raskt overblikk at: ved $n = 1000$ er kjøretiden $O = 1000 \cdot \log(1000) = 1000 \cdot 3 = 3000$

- **Hvordan er antall sammenlikninger og antall bytter korrelert med kjøretiden?**
 - Det ser tilsynelatende ut som de algoritmene som har mange sammenlikninger og bytter har høyere kjøretid enn de med færre
 - Kan også se på Figur 8 og Figur 9 at metodene som dominerer i kategoriene bytter. Tilsvarende tidligere konklusjoner er det quick og merge dominerer med bytter og sammenlikninger på lav input, og motsatt ved høy
 - La også merke til at Insertion og Selection har en viss sammenheng i antall sammenlikninger med n . I begge diagrammene ser det ut som antall sammenlikninger følger $n*(5*n/10)$, som gir 50 ved 10 input og 5000 ved 100 input
 - Merge og quick dominerer søylediagrammet for 10 input. Dette kan være fordi algoritmene bruker tid og sammenlikninger på å først sortere inputet slik at det senere raskt kan sorteres. Dette vil funke bedre på de lange inputene

- **Hvilke sorteringsalgoritmer utmerker seg positivt når n er veldig liten?**
 - Både insertion og selection utmerker seg godt med korte lister (Figur 3a og Figur 4). Algoritmene bruker lite tid på å strukturere og mellomlagre tallene i lista slik som i merge og quick.

- **Hvilke sorteringsalgoritmer utmerker seg positivt for de ulike input-filene?**
 - Hypotesen er at siden merge og quick har kjøretid $O(n*\log(n))$ skal disse være mest effektive for lengre lister, mens de to andre sorteringsalgoritmene insertion og selection skal fungere bedre på korte lister. Dette ser man godt at stemmer ved sortering av helt tilfeldige lister (Figur 4 og Figur 5)
 - Gjorde også en observasjon på at kjøretidsforskjellen mellom insertion og selection minket da input var tilfeldig (Figur 4) kontra nesten sortert (Figur 3b). Utifra observasjonene kan man se at insertion fungerer best på nesten sorterte lister. Dette støttes enda tydeligere opp ved Figur 6a kontra Figur 7. Her kan man se at selv om input er stort har insertion svært lik kjøretid som merge og quick.

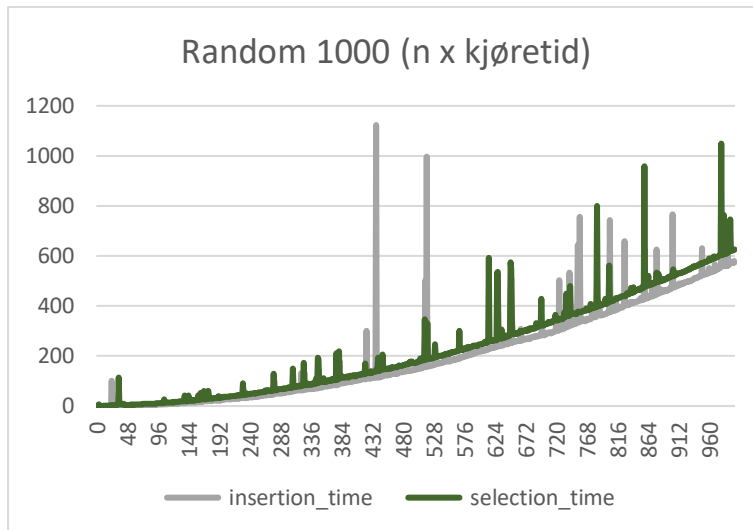
- **Har du noen overraskende funn å rapportere?**

- Merge sine hopp med plutselige høye kjøretider. Dette kan komme av at input-lista blir delt opp i to lister, hvor den ene lista er svært usortert, og den andre lista nesten er ferdig sortert. Da må det ene metodekallet vente på det andre før de kan flettes sammen.
- Insertion er overraskende god på større lister Fig. 3 b ($n = 100$). Dette støtter opp argumentet om at insertion fungerer svært godt på nesten ferdig sorterte lister
- Selection_swaps ved input 100 var svært lav, ved å se på koden konkluderte jeg med at det var fordi antall swaps var maks $n-1$

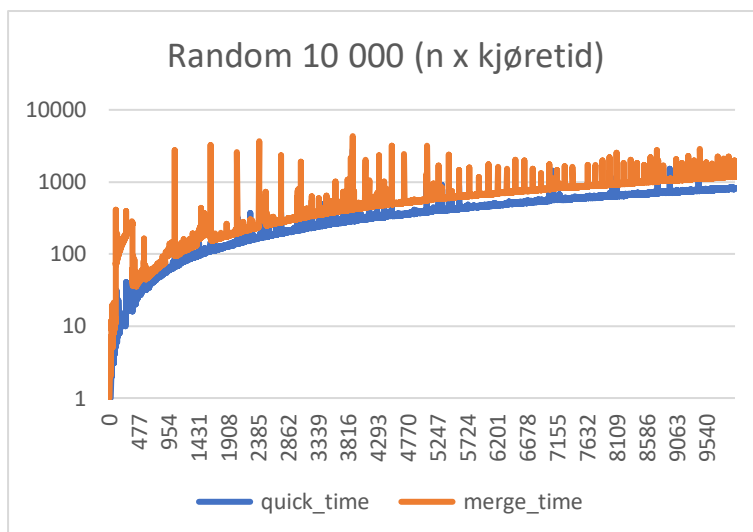
Figurer:

Enkelte steder har jeg laget to utgaver (a og b) av samme figur hvor jeg har tatt bort noen av resultatene for å lettere se kontrastene i grafene

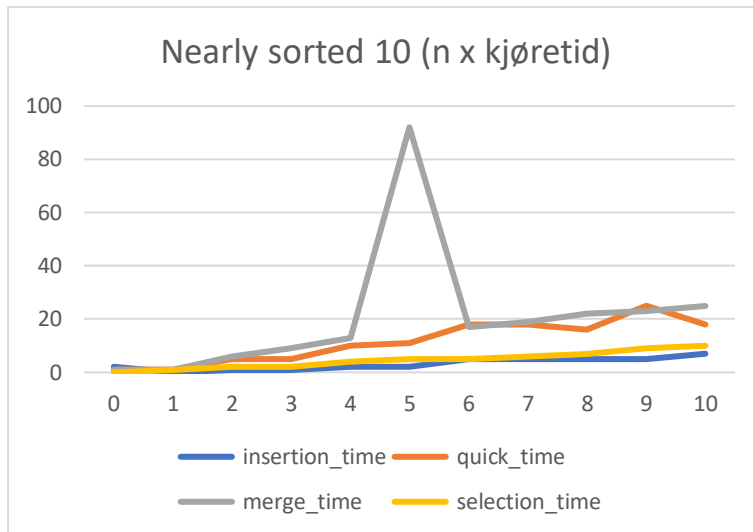
Figur 1:



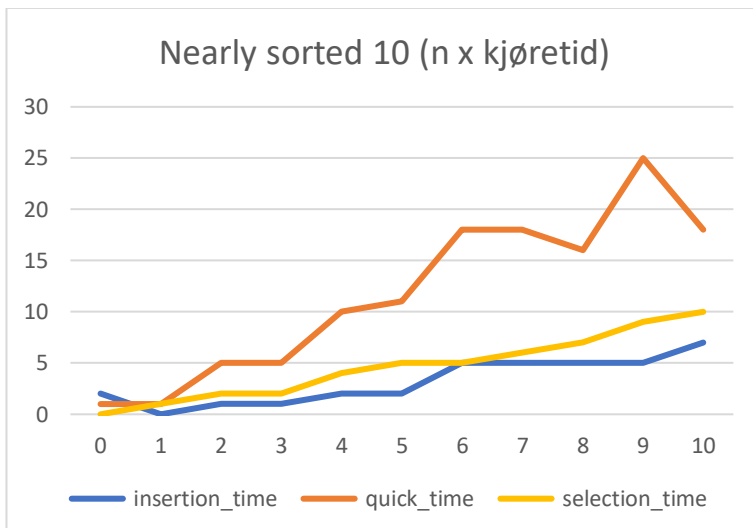
Figur 2:



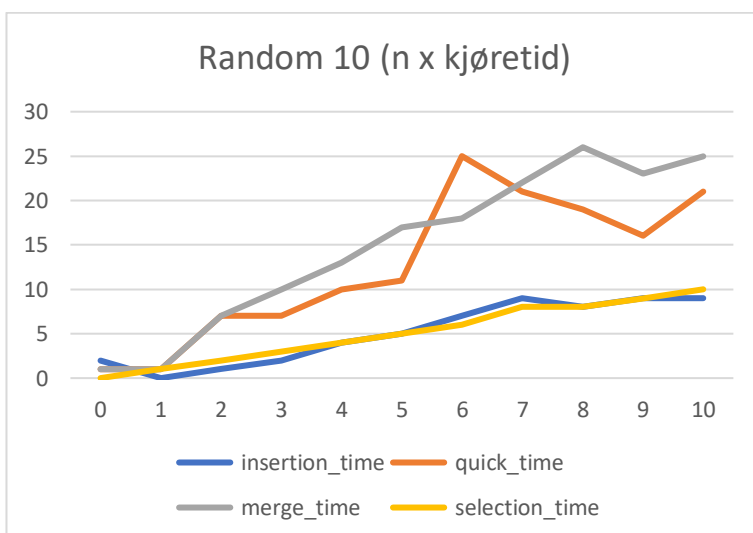
Figur 3 a:



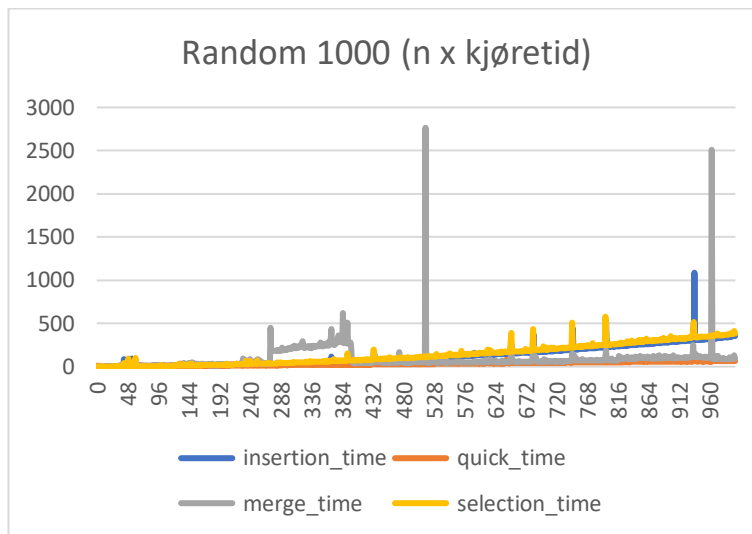
Figur 3 b:



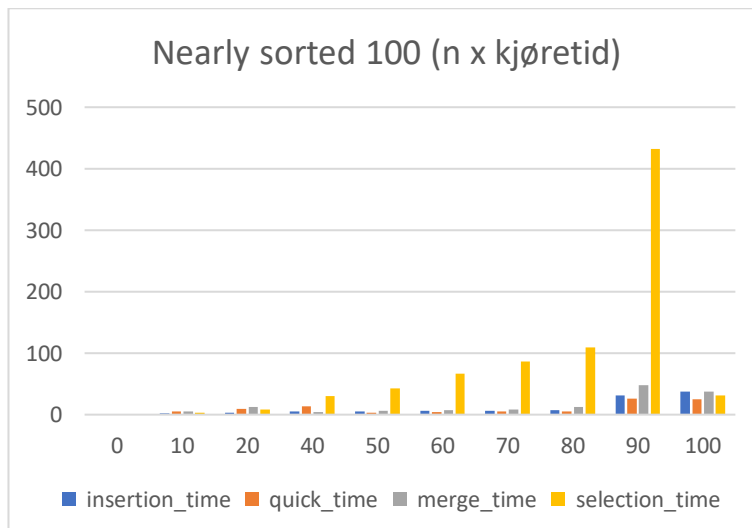
Figur 4:



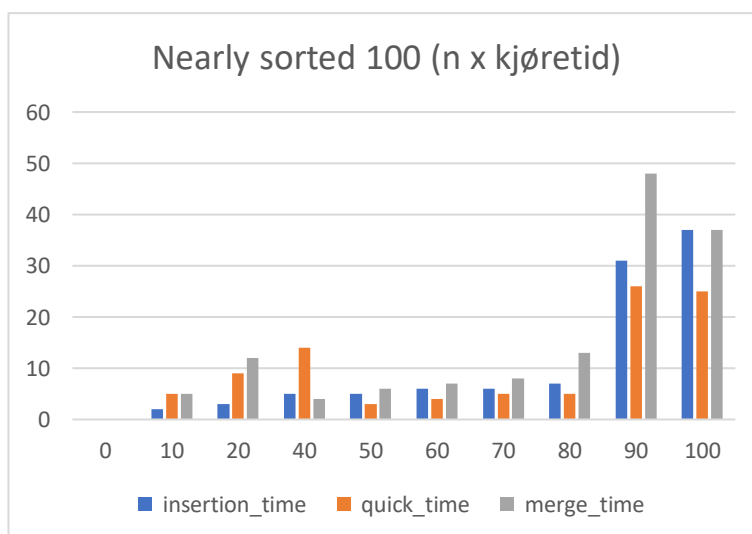
Figur 5:



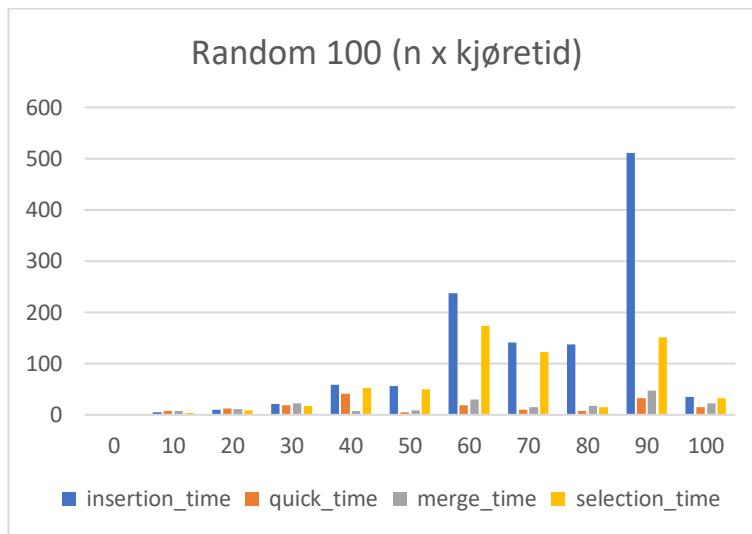
Figur 6a:



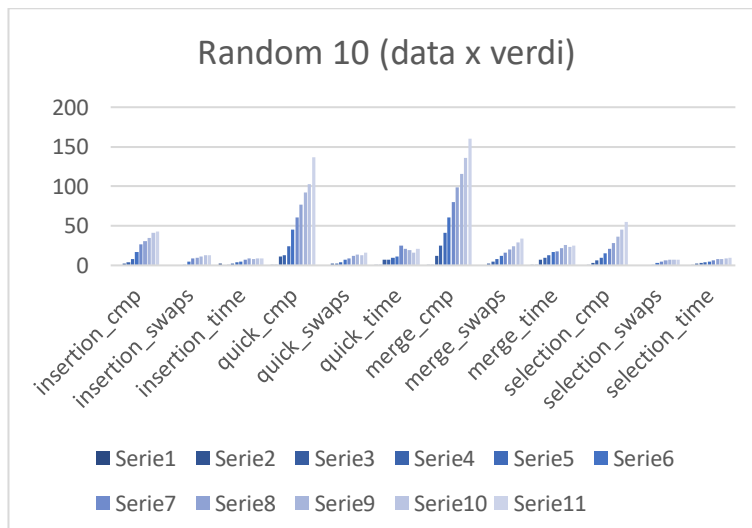
Figur 6b:



Figur 7:



Figur 8:



Figur 9:

